

Discord_Rag_Overview

Project Overview: Building a Discord RAG Bot

Welcome to your first AI project!

This assignment challenges you to build a **Discord Retrieval-Augmented Generation (RAG) Bot**. This bot will be capable of answering questions based on a specific knowledge base, going beyond what a standard Large Language Model (LLM) might know by "retrieving" relevant information before "generating" a response.

What is a RAG Bot? Imagine you want an AI to answer questions about this AI Internship program's internal documentation, or a training material. A regular LLM might hallucinate or not have the most up-to-date information. A RAG bot solves this by:

1. **Retrieval:** When a user asks a question, the bot first searches a dedicated knowledge base (e.g., a collection of documents) to find the most relevant pieces of information.
2. **Augmentation:** It then takes these retrieved pieces of information and "augments" the user's original query.
3. **Generation:** Finally, it sends this augmented query (original question + retrieved context) to an LLM, which uses the provided context to generate a more accurate and grounded answer.

Why Discord? Discord provides a familiar, real-world platform for deploying and interacting with your bot, allowing for easy testing and user feedback.

Various Role Involvement: Each role (Data Scientist, Frontend Engineer, Backend Engineer) has distinct responsibilities. See the tabs on the left side for role specific assignments.

Key Concepts to Understand:

- **Chunking:** Breaking down large documents into smaller, manageable pieces (chunks).
- **Embeddings:** Converting text chunks into numerical vectors, representing their semantic meaning.
- **Vector Database:** A specialized database optimized for storing and searching these vector embeddings (e.g., MongoDB Atlas Vector Search).
- **Vector Search:** Finding the most similar vectors (and thus, text chunks) to a given query vector.
- **LLMs:** Large Language Models, the "brain" that generates human-like text.
- **API Design:** How different parts of your application communicate with each other.
- **Deployment:** Getting your application running in a live environment.
- **Logging & Observability:** Monitoring your application's health and performance.
- **User Experience (UX) in AI:** Designing intuitive and effective ways for users to interact with AI.



Note: To assist with the development process, you are free to use any AI coding, feel free to use Azure AI Foundry and the free DeepSeek model.

- A. YT Deploy DeepSeek R1 using Azure AI Foundry and Build a Web Chatbot | No Charges for API Use = <https://www.youtube.com/watch?v=pj2knBX4S1w>
- B. MSFT = <https://azure.microsoft.com/en-us/pricing/purchase-options/azure-account> [Available only to new Azure customers = \$200 credit]
- C. MSFT Billing for Free Trial Account = <https://learn.microsoft.com/en-us/answers/questions/2283380/billing-for-free-trial-account>
- D. MSFT - DeepSeek R1 is now available on Azure AI Foundry and GitHub = <https://azure.microsoft.com/en-us/blog/deepseek-r1-is-now-available-on-azure-ai-foundry-and-github/>

Go through all of the links above so you have a good idea

Good luck, and remember to have fun learning!

Helpful Resources:

Here are some helpful resources that could be used in your Discord Chatbot journey.

- Youtube: [Code a Discord Bot with JavaScript](#)
- Youtube: [Code a Discord Bot with Python](#)
- Youtube: [Learn Discord.js V14](#)
- [Workshop created by Mentor Apporva](#) that walks through the end-to-end process of chunking, embedding, ingestion, implementing vector search, RAG, and adding memory from scratch, mostly without any frameworks, except LangChain for chunking
- Youtube: [Deploying a discord bot using docker](#)
- [Build a RAG System Using Claude 3 Opus And MongoDB](#)

Source Documents the Discord Chatbot should Reference:

1. [AI Bootcamp Journey & Learning Path](#) - Also shows high-level schedule
2. [Training For AI Engineer Interns](#) - Many video links in this document
3. [Intern FAQ - AI Bootcamp](#) = Frequently asked questions

Office Hours *(for weeks 1-3)*:

- Please look for announcements in Discord for exact dates/times of Office Hours.
- Use Office Hours to help with roadblocks rather than as a gatekeeper holding you back. This way you start coding and make progress while not waiting for scheduled weekly Office hour sessions.



Frontend Assignment

Frontend Engineer Assignment

As a Frontend Engineer, your role is crucial for how users interact with the Discord RAG bot. You will be responsible for designing and implementing a user-friendly and intuitive experience within the Discord environment. Your focus will be on the **user interface (UX) and interaction patterns**.

Phase 1: Preparation

Your primary goal is to **research user experiences in the agentic era, design a simple UX for the Discord bot, and identify how to collect user feedback**.

Actions:

1. Research AI UX & Agentic Experiences:

- Explore how leading AI products (e.g., ChatGPT, Google Bard, Microsoft Copilot, GitHub Copilot) design their user interactions.
- Focus on:
 - How users initiate conversations.
 - How the AI's responses are presented (e.g., formatting, clarity, length).
 - How the AI communicates its capabilities and limitations.
 - How users provide feedback (e.g., thumbs up/down, "regenerate," explicit feedback buttons).
 - How loading states and errors are handled.
 - **Crucially, consider how these patterns translate to a conversational interface like Discord.** Discord's native UI elements (messages, reactions, embeds, slash commands) will be your building blocks.
- *Some resources to start:*
 - "Designing Agentic AI Experiences" (various articles can be found via a quick search)
 - Case studies of existing Discord bots.

2. Design a Simple UX for the Discord Bot:

- Based on your research, sketch or wireframe a simple user flow for your Discord bot.
- Consider:
 - **Bot Invocation:** How will users "talk" to the bot? (e.g., direct messages, mentioning the bot with `@bot-name`, using slash commands like `/ask`).
 - **Question Input:** How will users ask questions?
 - **Response Display:** How will the bot's answers be formatted in Discord? (e.g., plain text, code blocks, Discord embeds for richer content).
 - **Handling Delays:** How will you indicate to the user that the bot is "thinking" or retrieving information?

- **Error Handling:** How will the bot communicate if it can't answer, or if an error occurs?
- **Feedback Mechanism:** This is a critical part of AI UX. How will users tell the bot if its answer was helpful or unhelpful? (e.g., emoji reactions like 👍/👎, a specific command like `/feedback "great answer!"`).

3. Identify User Feedback Needs:

- Determine what kind of feedback you want to collect (e.g., satisfaction with answer, relevance of retrieved information, suggestions for improvement).
- How will this feedback be captured and potentially sent to the backend/data scientists? (e.g., a simple API call with the user's reaction, text feedback).

Office Hours (optional):

Use Office Hours to help with roadblocks rather than as a gatekeeper holding you back. This way you can start developing your code to make progress and no longer wait for Office hours.

Some things to think about:

- Your findings on AI UX
- Your proposed Discord bot UX design (wireframes/sketches are encouraged).
- Your plan for collecting user feedback, including the specific Discord features you'll leverage (reactions, custom commands etc.).
- Any challenges you anticipate in implementing this UX with Discord.js and integrating with the backend.

Phase 2: Development

Can start development even if not attend Office hours. Start focusing on aligning on the UX, your focus shifts to implementing the Discord bot interface using ReactJS/NextJS.

Actions:

1. Discord Bot Setup (using `discord.js`):

- You will need to create a Discord application, set up a bot user, and get a bot token.
- You can use the `discord.js` library to interact with the Discord API. This will involve listening for messages, responding to them, and managing bot permissions.

2. Implement Discord Interaction Logic:

- Listen for specific messages (e.g., messages mentioning the bot, or slash commands).
- When a relevant message is received:

- Display a "thinking" or "typing..." indicator in Discord.
- Send the user's query to the backend API.
- Receive the response from the backend.
- Format and send the response back to the Discord channel using appropriate Discord.js methods (e.g., `channel.send`, `message.reply`, `embeds`).
- Implement the chosen **feedback mechanism**. For example, when the bot sends an answer, automatically add reaction emojis (👍, 👎), and listen for user clicks on these reactions to send feedback data to the backend.

3. Error and Loading States:

- Ensure the bot provides clear feedback to the user when it's processing a request or encounters an error.

Please note, there are a few helpful resource videos to reference in the main [Discord Rag Overview](#) page.

Backend Assignment

Backend Engineer Assignment

As a Backend Engineer, you are the bridge connecting the Discord bot (frontend interaction) with the RAG agent (data science logic). Your responsibilities include building robust APIs, handling scalability, deployment, and ensuring the application's reliability through logging and observability. Your focus will be on **system architecture, API design, and operational excellence**.

Phase 1: Preparation

Your primary goal is to **research deployment strategies, logging, observability, and propose a backend architecture** focused on these aspects.

Actions:

1. Backend Framework Research:

- Research Node.js/Express.js and Python (Flask/FastAPI) as choices for building web APIs. Understand their strengths and weaknesses for this type of application.
- Consider how to structure a backend application to support modularity (e.g., separate routes, controllers, services).

2. Deployment Strategies Research:

- Investigate different ways to deploy web applications and Discord bots.
- Optional: Potentially utilize **containerization with Docker**. Understand what Docker is, how to create a [Dockerfile](#), and why it's beneficial for deployment.
- Local deployment is the first step. Optional = Also, Look into cloud deployment options (e.g., Render, Heroku, AWS EC2, AWS Lambda, Google Cloud Run). Understand the basic pros and cons of each for a 24/7 running bot.

3. Logging & Observability Research:

- **Logging:** Research logging libraries/approaches for your chosen backend language. Understand different log levels (info, warn, error).
- **Observability:** Beyond simple logging, research basic observability concepts.
 - **Metrics:** How would you collect simple metrics (e.g., number of requests, response times, errors)?
 - **Monitoring:** What tools or services are used for monitoring application health? (e.g., Prometheus/Grafana, basic cloud provider monitoring).
 - **Tracing (optional, advanced):** Briefly understand distributed tracing for debugging complex systems.

4. API Design Considerations:

- Think about the API endpoints that will be needed:
 - To receive a user query from the Discord bot (frontend).

- To interact with the data scientist's RAG components.
- Potentially, an endpoint for document ingestion into the knowledge base.
- Potentially, an endpoint for receiving user feedback.

5. Architecture Diagram:

- Create a conceptual **backend architecture diagram** that integrates with the Data Scientist's RAG agent and the Frontend's Discord bot logic.
- Emphasize the deployment environment, logging setup, and how you envision monitoring working.
- Illustrate API communication flows between components.

Office Hours (optional):

Use Office Hours to help with roadblocks rather than as a gatekeeper holding you back. This way you can start developing your code to make progress and no longer wait for Office hours.

Some things to think about:

- Your proposed backend architecture diagram, highlighting deployment and observability aspects.
- Your chosen backend language/framework and justification.
- Your strategy for containerization with Docker.
- The logging library and approach you plan to use.
- How you plan to monitor the bot's health and performance.
- Your proposed API contracts/endpoints for communicating with the Data Scientists and the Frontend.

Phase 2: Development

Can start development even if not attend Office hours. Start focusing on implementing the backend APIs, integrating the RAG logic, and setting up logging/observability.

Actions:

1. API Implementation:

- Build the core API endpoints (e.g., `/api/rag-query`, `/api/feedback`) using your chosen backend framework (Express.js or Flask/FastAPI).
- These endpoints will receive requests from the Discord bot (or a proxy).
- Within the `/api/rag-query` endpoint, you will integrate with the Data Scientist's RAG logic (e.g., by importing Python modules directly if using Python, or making HTTP calls if the DS components are separate microservices).

2. Discord Bot Integration (if not handled by Frontend):

- Assuming a Frontend engineer decides to offload the direct `discord.js` interaction, you will implement the Discord bot logic within your backend. This means using `discord.js` to listen for messages and send responses.

3. Logging Setup:

- Integrate your chosen logging library.
- Log key events: incoming requests, RAG processing steps, LLM calls, responses sent, and errors.
- Ensure logs are structured and contain relevant information (e.g., user ID, query, timestamp, latency).

4. Observability Basics:

- Implement basic custom metrics (e.g., request count, error rate) that can be logged or exposed.
- Consider how you would set up simple dashboards if you had access to monitoring tools. For this assignment, focus on generating the data needed for monitoring.

5. Containerization (Docker):

- Create a `Dockerfile` for your backend application. This Dockerfile should define how to build an image containing your application and its dependencies.
- Test running your backend within a Docker container.

6. Collaboration with Data Scientists:

- If possible, reach out with a Data Scientist to understand the input/output requirements of their RAG components and integrate them seamlessly into your API.

Remember to start simple, and build up complexity iteratively. Good luck!

Please note, there are a few helpful resource videos to reference in the main [Discord_Rag_Overview](#) page.

Data Scientist Assignment

Data Scientist Assignment

As a Data Scientist, you are at the core of the RAG bot's "intelligence." You will be responsible for defining how the bot understands questions, retrieves information, and how the LLM ultimately uses that information to generate answers. Your focus will be on the **agent's logic and the data pipeline**.

Phase 1: Preparation

Your primary goal is to **research, understand, and propose an architecture** for the RAG agent, and a high-level workflow for its data.

Actions:

1. Deep Dive into RAG:

- Review the the provided MongoDB workshop notebook:
<https://github.com/mongodb-developer/genai-devday-notebooks/blob/main/notebooks/ai-rag-lab.ipynb>
- Understand each step: chunking, embedding, ingestion, vector search, RAG, and memory. Pay close attention to the "from scratch" approach.
- Research different **embedding models** (e.g., Sentence Transformers, OpenAI Embeddings, Cohere Embeddings, models from Hugging Face). Think through their strengths, weaknesses, and how they are typically used.
- Research **vector databases/stores** (e.g., Faiss for local prototyping, Pinecone, Weaviate, Milvus, and specifically **MongoDB Atlas Vector Search** as it aligns with the workshop's context).
- Familiarize yourself with basic concepts of **LLMs** and how they consume context.

2. Architecture & Workflow Diagram:

- Based on your research and the workshop, design a conceptual **architecture diagram** for the RAG agent. This should illustrate the major components and how they interact.
- Create a **workflow diagram** showing the flow of data when a user asks a question to the Discord bot, through your RAG system, and back to the user.
- Consider:
 - Where will the knowledge base reside?
 - How will documents be chunked and embedded?
 - Where will embeddings be stored?
 - How will a user query be embedded and used for search?
 - How will retrieved context be passed to an LLM?

- Which LLM will you propose (e.g., a free/open-source option for a PoC like `ollama` with a local model, or a cloud API like Gemini, OpenAI if you get access)?

Office Hours (optional):

Use Office Hours to help with roadblocks rather than as a gatekeeper holding you back. This way you can start developing your code to make progress and no longer wait for Office hours.

Some things to think about:

- Your proposed RAG agent architecture diagram.
- Your RAG workflow diagram.
- Your rationale for choosing specific embedding models, vector store approaches, and LLM providers.
- Any challenges or open questions you've identified (e.g., how to handle large documents, real-time updates to the knowledge base, cost implications of LLMs).
- How you plan to collaborate with the Backend Engineers on API design for your RAG components.

Phase 2: Development

Can start development even if not attend Office hours. Start focusing on implementing the core RAG logic.

Actions:

- 1. Data Ingestion Pipeline:**
 - Implement the process of **chunking** your knowledge base documents.
 - Generate **embeddings** for each chunk.
 - **Ingest** these embeddings into your chosen vector store (e.g., a simple in-memory list for initial prototyping, a local vector store like FAISS, or integrate with MongoDB Atlas Vector Search)
- 2. Retrieval Logic:**
 - Implement the logic to convert an incoming user query into an embedding.
 - Perform a **vector search** against your stored embeddings to retrieve the most relevant chunks.
- 3. Augmentation & Generation (RAG Chain):**
 - Construct the **prompt** for the LLM by combining the user's original query and the retrieved context.

- Make an API call to the chosen LLM to generate the final answer.

4. **Collaborate with Backend Engineers** (*Time Permitting*):

- If time allows: Work closely with the Backend Engineers to define the **API endpoints** your RAG components will expose (e.g., an endpoint to receive a query and return an answer, or an endpoint for document ingestion).

5. **Agent Evaluation** (*Time Permitting*):

- If time allows, research and implement basic metrics for evaluating your RAG system's performance. Consider:
 - **Relevance/Precision:** How often are the retrieved chunks actually relevant to the query?
 - **Faithfulness:** Does the generated answer only use information from the retrieved context, or does it hallucinate?
 - **Answer Correctness:** Is the final answer factually accurate based on the knowledge base?
- You might collect a small set of sample questions and their expected answers from your knowledge base to test against.

Remember to start simple and iterate. Focus on getting a basic end-to-end flow working before optimizing or adding complex features.